Security Program Maturity Review & Lessons Learned

Current-state maturity, target state, lessons learned, and next-step recommendations

Capability	Current maturity	90-day target	Key improvement
Governance & risk	Developing	Defined	Risk owners, review cadence, evidence-based closure
IAM	Developing	Defined	MFA, RBAC, JML, privileged recertification
Vulnerability mgmt	Developing	Defined	Asset ownership, SLAs, exceptions, validation
SOC monitoring	Initial / Developing	Defined	P1 logs, detections, triage runbooks, metrics
Incident response	Developing	Defined	Severity, playbooks, evidence, tabletop, PIR
Recovery	Developing	Defined	Restore testing and documented RTO/RPO evidence
Data protection	Developing	Defined	Classification, DLP workflow, access governance
Third-party risk	Initial / Developing	Developing	Vendor tiers, recurring reviews, access expiry

Lessons Learned

- A security program is stronger when controls have explicit owners, evidence, validation, and escalation rules.
- Centralized visibility is a dependency for both detection and audit readiness.
- Severity and risk scores should drive prioritization, but business context determines the final decision.
- Automation reduces repetitive work only when inputs, logging, testing, rollback, and ownership are well designed.
- Portfolio evidence is most credible when simulated facts are labeled clearly and framework alignment is not confused with formal compliance.

Next 6-Month Enhancements

- Expand detection content to cloud-native and SaaS administrative behaviors.
- Add recurring configuration-drift checks for Linux and cloud security baselines.
- Create a third-party risk scoring model tied to data access and business criticality.
- Add quarterly tabletop and restore-test cadence with trend reporting.
- Track control effectiveness through failed tests, repeat incidents, overdue remediation, and exception aging.